

Test Plan

Personal Project

C. Stijlaart

Table of Contents

1. Introduction	1
1.1. Purpose	1
1.2. Scope	1
1.3. Objectives	1
2. Test Items	2
3. Test Cases	3
3.1. Collect Environmental Data (UC01)	3
3.2. Generate Musical Composition (UC02)	3
3.3. Change Musical Composition (UC03)	3
4. Software testing	5
4.1. Unit Testing (Google test)	5
4.2. Component testing	5
4.3. Black box testing	5

1. Introduction

1.1. Purpose

This test plan outlines the testing approach for the embedded system designed to transform environmental data into musical compositions. It covers the verification of user, functional, and non-functional requirements outlined in the System Design Document.

1.2. Scope

The testing will focus on validating the key functionalities of the system, including environmental data collection, musical composition generation, algorithm switching, and database interactions.

1.3. Objectives

- Verify that the system collects environmental data accurately.
- Ensure the correct generation of musical compositions based on the collected data.
- Confirm the ability to switch algorithms for musical composition.
- Validate proper storage and retrieval of data in/from the database.

2. Test Items

1. Controller Unit (CU): The central processing unit for the system.
2. Environmental Data Unit (EDU): Collects data from the system's environment.
3. Database (DB): Stores and manages data generated by the system.
4. Interface (IF): Provides an application interface for external interactions.
5. Raspberry Pi 4: Responsible for making decisions and generating musical compositions.
6. Arduino Nano 33 IoT: Collects environmental data.
7. Environmental Sensors: Various sensors used to measure data.
8. Speakers: Output device for the generated musical composition.

3. Test Cases

3.1. Collect Environmental Data (UC01)

3.1.1. Test Case 4.1.1: Verify Data Collection

1. **Description:** Confirm that the system starts collecting environmental data at startup.
2. **Procedure:** Power on the system and observe data collection status.
3. **Expected Result:** Environmental data collection starts without errors.
4. **Type of Testing:** Unit Testing (for data collection functionality)

3.1.2. Test Case 4.1.2: Handling No Connected Sensors

1. **Description:** Test the system's response when attempting to collect data without connected sensors.
2. **Procedure:** Disconnect sensors, initiate data collection, and observe the system's response.
3. **Expected Result:** System provides an appropriate error message.
4. **Type of Testing:** Negative Testing (for error handling)

3.2. Generate Musical Composition (UC02)

3.2.1. Test Case 4.2.1: Verify Musical Composition Audibility

1. **Description:** Confirm that the system generates a musical composition that is audible over the speakers.
2. **Procedure:** Request a musical composition through the interface and listen for the output over the speakers.
3. **Expected Result:** A unique musical composition is generated and can be heard clearly over the speakers.
4. **Type of Testing:** Component Testing (for composition generation and audible output)

3.2.2. Test Case 4.2.2: Handling No Collected Data

Description: Test the system's response when attempting to generate a composition without collected data. **Procedure:** Clear collected data, request a musical composition, and observe the system's response. **Expected Result:** System provides an appropriate error message. **Type of Testing:** Negative Testing (for error handling)

3.3. Change Musical Composition (UC03)

3.3.1. Test Case 4.3.1: Verify Algorithm Switching

1. **Description:** Validate the system's ability to change the musical composition algorithm through the interface.
2. **Procedure:** Change the algorithm through the interface and observe the resulting musical composition.
3. **Expected Result:** The musical composition reflects the selected algorithm.
4. **Type of Testing:** Component Testing (for algorithm.switching functionality)

4. Software testing

As mentioned previously, there is made use of multiple types of testing. Each kind tests a specific aspect of the system. The different kinds of testing that are performed, are discussed throughout this chapter.

4.1. Unit Testing (Google test)

For each part of the system there are separate google tests written. These google tests are written in such a way that it will be easy to figure out the problem when the system unexpectedly breaks. To optimise the functionality of the tests, they are added to the git and are build during the pipeline. If the tests fail, the branch will not be able to merge.

4.2. Component testing

Certain aspects of the system can not be verified with google tests. Therefore, component testing is used. Component testing focuses on one component and its input/output. When this form of testing is performed, a specific component will be isolated and run through some functional tests. These functional tests help verify that the component works on its own regarding their main purpose.

4.3. Black box testing

Black box testing is used verify the functionality of the system based on the defined requirements. There are multiple kind of black box testing, the one that will be used for our purpose are funtional testing.